



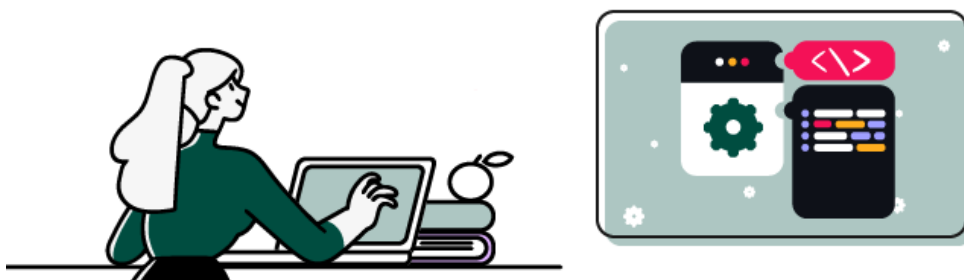
FS



← Découvrez les librairies Python pour la Data Science

25% de progression

Fusionnez des données avec Pandas



Vous êtes à présent à même d'agréger vos données. Mais que faire lorsque ces dernières sont partagées entre plusieurs fichiers de données et donc, plusieurs data frames ? C'est ce que je vous propose de découvrir !

Comprenez l'intérêt de la fusion de données

Dans une banque, les sources de données peuvent être multiples. Nous avons déjà notre liste de clients d'une part, et la liste des prêts fournie par le service adéquat au sein de notre établissement. Mais il peut y avoir bien d'autres choses ! On peut imaginer notamment :

- les informations issues de la navigation des clients sur le site web ;
- les données récoltées via notre application mobile ;
- les données issues des différents services (Professionnels, Investissement) ;
- etc.

Chaque source peut être matérialisée par un ou plusieurs fichiers de données et donc, un ou plusieurs data frames. On peut donc être amenés à travailler avec de nombreuses sources différentes, qui ont pourtant des informations complémentaires.

Prenons par exemple nos deux fichiers de données, **clients** et **prêts** :

- d'un côté, nous avons des informations générales sur nos clients, comme leur e-mail ou leur genre ;
- de l'autre côté, nous avons des informations sur les prêts qui ont



Ces informations sont complémentaires car elles concernent **les mêmes clients**. Il serait donc intéressant de pouvoir **recouper** ces deux jeux de données pour mener des analyses pertinentes, comme par exemple vérifier si les montants d'emprunt ou les taux attribués par nos conseillers varient en fonction du genre de notre client.

Le fait de rassembler comme cela plusieurs jeux de données différents est appelé une **jointure**, en algèbre relationnelle.

Tout comme l'agrégation, c'est une opération très courante lorsqu'on manipule des données, car il est rare que l'ensemble des données à utiliser proviennent d'une seule et unique source !

Pour effectuer une jointure, nous aurons absolument besoin d'une information qui soit en commun dans chaque jeu de données : c'est que nous appelons **une clé**. La clé à utiliser pour faire une jointure peut être formée d'une (comme c'est très souvent le cas) ou plusieurs colonnes : il n'y a pas de limite à ce niveau ! La seule condition est que la clé choisie soit présente dans chaque jeu de données qu'on essaie de joindre aux autres.

Dans l'exemple ci-dessus, l'opération de jointure a un sens car il est possible de faire le lien entre les informations générales de clientèle et les informations de prêt, à partir de l'identifiant client : cette information est commune aux deux data frames. C'est donc la clé que nous allons utiliser pour effectuer notre jointure.

Comme pour l'agrégation, l'opération est quasi équivalente à une jointure qu'on pourrait réaliser en SQL, à la différence près que les clés primaires et étrangères ne sont pas explicitement identifiées dans un data frame Python.

Je vous propose de voir comment réaliser cela avec Python.

Fusionnez des données avec `merge`

La fonction ou méthode prévue par Pandas pour faire une jointure se



méthode. Car il existe deux façons de faire une jointure entre 2 data frames `A` et `B` avec Pandas :

- via la fonction Pandas. Dans ce cas-là, l'écriture sera `pd.merge(A, B)` ;
- via la méthode de data frame. Dans ce cas-là, l'écriture sera `A.merge(B)` .

Quelles sont donc les différences entre ces deux écritures ?

Absolument aucune, en dehors de la syntaxe présentée ci-dessus. Que vous utilisiez la fonction ou la méthode, vous aurez à paramétrer les mêmes arguments et vous obtiendrez le même résultat ! Disons que vous pouvez choisir l'écriture qui vous semble la plus logique et la plus simple à comprendre.

Mais il n'y avait pas une histoire de clé, ou bien j'ai manqué quelque chose ?

Dans ce que je vous ai présenté jusque-là, j'ai totalement occulté la clé utilisée pour faire la jointure. Par défaut, si on ne lui indique rien de spécifique, Pandas va chercher les colonnes en commun dans les différents data frames (celles qui portent le même nom) et va les sélectionner comme clés : c'est ce qu'on appelle **une jointure naturelle**.

Mais nous pouvons naturellement spécifier la clé à utiliser ! Il va donc y avoir deux cas possibles :

- lorsque la clé porte le même nom dans chaque data frame, on peut utiliser l'argument `on` ;
- lorsque la clé n'a pas le même nom, il est nécessaire de préciser

quelle clé on va utiliser dans chaque data frame. On aura alors besoin des arguments `right_on` et `left_on` qui correspondront à la clé dans le data frame à droite et celle dans le data frame à gauche.



(gauche) revient régulièrement lorsqu'on parle de jointure. Laissez-moi expliquer ça : c'est simplement l'ordre dans lequel nous plaçons nos data frames au sein du `merge` .

Voici 3 exemples de jointures :

```
1 # jointure entre 2 dataframes A et B
2 pd.merge(A, B, on='id')
```

```
1 # jointure entre 2 dataframes A et C
2 C.merge(A, left_on='identifiant', right_on='id')
```

```
1 # jointure entre 2 dataframes A et B
2 pd.merge(A, B, left_on='id', right_on='identifiant')
```

Décomposons chaque cas :



frame B, une colonne qui porte ce nom. Cela renverra une erreur dans le cas contraire ;

- dans le cas suivant, le data frame considéré comme **left** est le data frame sur lequel on effectue la jointure : ici, il s'agit donc du data frame C. Le data frame A est considéré comme étant à droite. La jointure est réalisée sur la clé `identifiant` pour le data frame de gauche (ici, C), et `id` pour l'autre ;
- dans le dernier cas, le data frame considéré comme **left** est le premier renseigné, donc ici A. B est le data frame **right**. La jointure est réalisée avec la clé `id` du data frame de gauche (ici, A), et `identifiant` du data frame de droite.

Je vous propose à présent de joindre nos deux data frames `prêts` et `clients` dans un nouveau data frame nommé très originalement... `data` . La clé commune à chaque data frame est l'identifiant, c'est donc ce que nous utiliserons comme clé pour effectuer notre jointure :

```
1 data = pd.merge(clients, prêts, on='identifiant')
2 display(data)
```

Voilà ! Nous avons à présent un super data frame contenant TOUTES nos informat... Hop hop hop, pas si vite !

Vous ne l'avez peut-être pas remarqué, mais nous avons perdu des informations de prêts lors de notre jointure. En effet, nous avons 244 prêts attribués et répertoriés dans le fichier `prêts.csv`. Or, il ne reste *que* 162 lignes à l'issue de notre jointure... Où sont donc passées nos lignes disparues ?

Il y a un dernier aspect sur les jointures que nous n'avons pas encore abordé, ce sont **les types de jointures**. Et vous vous en doutez très certainement, mais oui, c'est *à cause d'elles* que certaines de nos

lignes ont disparu.

Explorez les différents types de jointure

Lorsqu'on effectue des jointures avec Pandas, il est indispensable de



Pandas doit *traiter* les différentes clés de nos data frames, notamment lorsqu'il y a des soucis de correspondance – une clé présente dans un data frame mais pas dans l'autre.

Il existe 4 types de jointures, qui sont toutes focalisées sur les différentes clés :

- interne ;
- à gauche ;
- à droite ;
- externe.

Regardons chaque type en détail, en considérant les deux data frames suivants, `taille` et `poids`, dans lesquels nous avons la taille et le poids de certains individus :

	id	Taille
0	1	1.80
1	2	1.70
2	3	1.75

	id	Poids
0	1	80
1	3	75
2	5	70

La **jointure interne** (ou **inner join**, en anglais) est le type de jointure par défaut. Nous n'avons d'ailleurs fait que des jointures internes

jusque-là. Avec une jointure interne, ne sont conservées dans le résultat final que les lignes dont les clés sont dans le premier data frame ET dans le second data frame.

C'est-à-dire que pour chaque clé identifiée à droite ou à gauche, elle



elle conserve la clé avec les informations des autres tables, sinon, elle supprime la clé dans le résultat final. Le code pour effectuer cette jointure est :

```
pd.merge(taille, poids, on='id', how='inner')
```

	id	taille	poids
0	1	1.80	80
1	3	1.75	75

Exemple de jointure interne (inner join)

Nous n'avons à la fin que 2 résultats, pour les identifiants 1 et 3 : ce sont en effet les seuls qui sont présents dans les deux tables. 2 est présent dans *taille* mais pas dans *poids*, et 5 est présent dans *poids*, mais pas dans *taille*.

La **jointure à gauche** (ou **left join**) est une jointure qui se concentre sur les identifiants de la table située à gauche (ici, *taille*). C'est-à-dire qu'elle conserve **forcément** toutes les clés qui se trouvent dans la première table, et complète, lorsqu'elle le peut, avec les informations de la seconde table.

Et comment faire lorsqu'il n'y a pas de correspondance ?

Ah, ça... Elle complète avec ce qu'on appelle des **valeurs manquantes**. C'est une valeur par défaut, qui ne représente rien, et qui est placée lorsqu'une information est manquante. Regardons le résultat ensemble, pour rendre cela un peu plus clair :

```
pd.merge(taille, poids, on='id', how='left')
```

	id	taille	poids
0	1	1.80	80.0



2	3	1.75	75.0
---	---	------	------

Exemple de jointure à gauche (left join)

Les identifiants 1 et 3 ont des correspondances, donc ils ne posent pas de problème. En revanche, l'identifiant 2 n'apparaît que dans le premier data frame. Il n'est donc pas possible de trouver l'information "poids" pour l'individu ayant pour identifiant 2 ! **L'information est manquante**. Elle est donc matérialisée par Pandas par une valeur significative : **NaN**, qui est l'abréviation de **Not a Number**. C'est cette valeur qui représente l'ensemble des valeurs manquantes avec Pandas.

Ces valeurs manquantes posent une vraie problématique en analyse de données. Les différentes solutions pour gérer ces valeurs font l'objet d'**un autre cours** que je vous invite à consulter une fois que vous aurez fini celui-ci, si le sujet vous intéresse !

La jointure à droite (ou **right join**) est... l'alter ego de la jointure à gauche, avec le data frame de droite ! Voilà le code correspondant :

```
pd.merge(taille, poids, on='id', how='right')
```

	id	taille	poids
0	1	1.80	80
1	3	1.75	75
2	5	NaN	70

Exemple de jointure à droite (right join)

Enfin, la **jointure externe** (ou **outer join**) est une sorte de conjonction des jointures à gauche et à droite. C'est-à-dire qu'on conserve

TOUTES les clés trouvées, que ce soit à gauche ou à droite, et on *bouche* les informations manquantes par des valeurs manquantes :

```
pd.merge(taille, poids, on='id', how='outer')
```



0	1	1.80	80.0
1	2	1.70	NaN
2	3	1.75	75.0
3	5	NaN	70.0

Exemple de jointure externe (outer join)

Décomposons le résultat :

- les identifiants 1 et 3 ont des correspondances, donc l'ensemble des informations sont correctement renseignées ;
- les identifiants 2 et 5 ne sont présents que dans un seul data frame (taille pour l'id 2 et poids pour l'id 5). Ainsi, les informations manquantes ont été complétées par des **NaN** .

Si certaines données ont disparu lors de la jointure entre nos clients et nos prêts, c'est que certains identifiants manquent dans un des deux fichiers. Dans les faits, il manque des informations, dans le fichier clients, qui se trouvent en réalité dans un autre fichier. Vous pourriez mener successivement deux jointures, mais il existe une autre alternative : la **concaténation**.

Concaténez des données avec Pandas

La **concaténation** intervient lorsqu'on souhaite assembler plusieurs data frames qui ont la même structure, les mêmes colonnes, les mêmes types, mais des informations différentes. Le cas typique correspond à des exports de données mensuelles, où on aurait les mêmes variables d'un mois à l'autre.

C'est le cas également ici ! Nous avons deux fichiers de clientèle, qui ont la même structure, mais des clients différents. L'idée ici ne serait donc pas de joindre les deux fichiers, mais simplement de les mettre l'un à la suite des autres, comme lors d'un ajout d'élément à une liste,

via la méthode `append` . La jointure a une dimension plus horizontale, alors que la concaténation est plus verticale.

La fonction Pandas permettant de faire une concaténation est la fonction `concat` . Pour concaténer plusieurs data frames, il suffit de



`concat` sur cette liste.

Par exemple, voilà comment procéder avec deux data frames, `df1` et `df2` :

```
1 liste_concat = [df1, df2]
2 pd.concat(liste_concat)
```

Le souci d'une concaténation, c'est qu'elle ne gère pas du tout les index par défaut. L'opération met juste le second data frame à la suite du premier, sans appliquer la moindre modification. Ainsi, vous aurez des index en doublon, ce qui peut être problématique ! Pour éviter cela, on peut utiliser l'argument `ignore_index` , qui équivaut à appliquer une méthode `reset_index` :

```
pd.concat([df1, df2], ignore_index=True) .
```

Je vous propose de mettre toutes ces fonctions et méthodes en application, en essayant de rassembler les données de nos 3 fichiers de données dans l'exercice suivant.

À vous de jouer



Contexte

Nos informations sont partagées dans plusieurs fichiers. En effet, nous avons d'une part le fichier contenant l'ensemble des informations de prêts, que nous avons déjà utilisé à plusieurs reprises. Nous avons



dans un, mais dans deux autres fichiers.

Il serait bien plus pratique de pouvoir rassembler l'ensemble de ces fichiers en un seul.

Consignes

Votre tâche cette fois-ci va être de rassembler ces différents fichiers pour ouvrir les possibilités de traitement, d'analyse, etc. Pour cela, vous devez utiliser l'ensemble des jeux de données à votre disposition (les deux fichiers clients et le fichier de prêts), et appliquer les différentes méthodes Pandas présentées dans cette partie, pour les fusionner.

Prêt ? C'est parti pour l'[exercice](#) !

Vérifiez votre travail

Que diriez-vous de comparer vos réponses avec [la correction](#), à présent ?

En résumé

- Il existe deux façons de *fusionner* deux data frames :
 - si les data frames ont la même structure, on peut faire une concaténation via la fonction `concat` : mettre les 2 data frames bout à bout ;
 - sinon, on peut faire une jointure via la fonction/méthode `merge` .
- Une jointure permet de joindre les informations de 2 data frames à partir d'une clé, une variable commune aux 2 data frames.
- Il existe 4 types de jointures :
 - la jointure interne qui conserve les clés se trouvant dans le

premier ET le second data frame ;

- la jointure à droite (ou à gauche) qui conserve uniquement les clés se trouvant dans le data frame à droite (ou à gauche), et complète les informations manquantes par des



- la jointure externe qui conserve toutes les clés se trouvant dans le premier OU le second data frame.

Nous sommes maintenant à même de pouvoir effectuer de nombreuses manipulations sur nos data frames. Je vous invite à présent à tester les connaissances acquises avec le prochain quiz !



[Avez-vous une suggestion pour nous ?](#) 

Faire le quiz →